MPT Table (Merkle Patricia Trie Table)

목적

MPT Table은 **이더리움 상태 트리의 업데이트를 검증**하는 테이블입니다. 이 테이블은 MPT Circuit에서 생성되며, State Circuit에서 상태 변경에 대한 Merkle 증명을 검증하는 데 사용됩니다. Scroll은 zkTrie (Sparse Binary Merkle Patricia Trie)를 사용하여 Poseidon 해시 기반의 효율적인 증명을 제공합니다.

테이블 스키마

파일: `zkevm-circuits/src/table.rs` (line 788)

```
pub struct MptTable {
/// 행 활성화 여부
pub q_enable: Column<Fixed>,
/// 계정 주소
pub address: Column<Advice>,
/// 스토리지 키
pub storage_key: Column<Advice>,
/// 증명 타입 (MPTProofType)
pub proof_type: Column<Advice>,
/// 새 루트 해시
pub new_root: Column<Advice>,
```

```
/// 이전 루트 해시
pub old_root: Column<Advice>,
/// 새 값
pub new_value: Column<Advice>,
/// 이전 값
pub old_value: Column<Advice>,
}
```

컬럼 설명

1. q_enable (Fixed)

- 해당 행이 유효한 MPT 행인지 표시

2. address (Advice)

- 계정 주소 (20바이트)
- 계정 트리 업데이트 시 사용

3. storage_key (Advice, SecondPhase)

- 스토리지 키 (32바이트, RLC 인코딩)
- 스토리지 트리 업데이트 시 사용
- SecondPhase에 할당

4. proof_type (Advice)

- 증명 타입을 식별하는 태그 (`MPTProofType`)
- 계정 필드 또는 스토리지 업데이트 타입

5. new_root (Advice, SecondPhase)

- 업데이트 후 트리의 루트 해시
- Poseidon 해시 (RLC 인코딩)

6. old_root (Advice, SecondPhase)

- 업데이트 전 트리의 루트 해시
- Poseidon 해시 (RLC 인코딩)

7. new_value (Advice, SecondPhase)

- 업데이트 후 값
- 계정 필드 또는 스토리지 값

8. old_value (Advice, SecondPhase)

- 업데이트 전 값
- 계정 필드 또는 스토리지 값

MPTProofType 열거형

파일: `mpt-circuits/src/types.rs`

```
pub  enum MPTProofType {
// 계정 필드 업데이트
NonceChanged = 1, // Nonce 변경
BalanceChanged, // Balance 변경
CodeHashExists, // Keccak Code Hash 존재
PoseidonCodeHashExists, // Poseidon Code Hash 존재
CodeSizeExists, // Code Size 존재
```

```
AccountDoesNotExist, // 계정 비존재

// 스토리지 업데이트

StorageChanged, // 스토리지 값 변경

StorageDoesNotExist, // 스토리지 키 비존재

// 계정 삭제

AccountDestructed, // 계정 삭제 (SELFDESTRUCT)

}
```

증명 타입 설명

계정 필드 증명

1. NonceChanged

- **의미**: 계정 nonce 변경
- **사용**: 트랜잭션 실행, 컨트랙트 생성
- **값**: nonce (uint64)

2. BalanceChanged

- **의미**: 계정 잔액 변경
- **사용**: ETH 전송, 가스 지불
- **값**: balance (uint256)

3. CodeHashExists

- **의미**: Keccak code hash 존재 확인
- **사용**: 컨트랙트 코드 검증
- **값**: keccak_code_hash (32바이트)

4. PoseidonCodeHashExists

- **의미**: Poseidon code hash 존재 확인
- **사용**: zkTrie에서 코드 검증
- **값**: poseidon_code_hash (32바이트)

5. CodeSizeExists

- **의미**: 코드 크기 존재 확인
- **사용**: EXTCODESIZE opcode
- **값**: code_size (uint64)

6. AccountDoesNotExist

- **의미**: 계정이 존재하지 않음
- **사용**: 새 계정 생성 전 확인
- **값**: 0

스토리지 증명

7. StorageChanged

- **의미**: 스토리지 값 변경
- **사용**: SSTORE opcode
- **값**: storage_value (uint256)

8. StorageDoesNotExist

- **의미**: 스토리지 키가 존재하지 않음
- **사용**: 새 스토리지 슬롯 생성 전 확인
- **값**: 0

계정 삭제

9. AccountDestructed

- **의미**: 계정 삭제
- **사용**: SELFDESTRUCT opcode
- **값**: 0

AccountFieldTag와 MPTProofType 매핑

파일: `zkevm-circuits/src/table.rs` (line 774)

```
impl From<AccountFieldTag> for MPTProofType {
fn  from(tag: AccountFieldTag) -> Self {
match  tag {
AccountFieldTag::Nonce => Self::NonceChanged,
AccountFieldTag::Balance => Self::BalanceChanged,
AccountFieldTag::KeccakCodeHash => Self::CodeHashExists,
AccountFieldTag::CodeHash => Self::PoseidonCodeHashExists,
AccountFieldTag::NonExisting => Self::AccountDoesNotExist,
AccountFieldTag::CodeSize => Self::CodeSizeExists,
}
}
}
```

이 매핑을 통해 RW Table의 `AccountFieldTag` 를 MPT Table의 `MPTProofType` 으로 변환합니다.

zkTrie 구조

Scroll은 **zkTrie**를 사용합니다. 이는 Sparse Binary Merkle Patricia Trie로, Poseidon 해시를 사용하여 효율적인 영지식 증명을 제공합니다.

특징

1. **Binary Tree**: 각 노드는 최대 2개의 자식 (left, right)
2. **Sparse**: 대부분의 노드가 비어있음 (효율적인 저장)
3. **Poseidon Hash**: SNARK-friendly 해시 함수
4. **Path Compression**: 긴 경로를 압축하여 저장

트리 구조

```
Root
/ \
/ \
Node1 Node2
/ \ / \
Leaf1 Leaf2 ... LeafN
```

Leaf 노드:

- 계정 트리: 계정 정보 (nonce, balance, code_hash, code_size)
- 스토리지 트리: 스토리지 값

테이블 레이아웃

계정 업데이트

예시: 계정 잔액 변경

```
address | storage_key | proof_type | new_root | old_root | new_value | old_value

---------|-------------|-----------------|----------|----------|-----------|----

0x1234.. | 0 | BalanceChanged | root2 | root1 | 100 ETH | 50 ETH
```

스토리지 업데이트

예시: 스토리지 값 변경

```
address | storage_key | proof_type | new_root | old_root | new_value | old_value

---------|-------------|-----------------|----------|----------|-----------|----

0x1234.. | key_rlc | StorageChanged | root2 | root1 | 42 | 0
```

계정 생성

예시: 새 계정 생성

```
address | storage_key | proof_type | new_root | old_root | new_value | old_value

---------|-------------|----------------------|----------|----------|-----------

0x5678.. | 0 | AccountDoesNotExist | root1 | root1 | 0 | 0

0x5678.. | 0 | NonceChanged | root2 | root1 | 0 | 0

0x5678.. | 0 | BalanceChanged | root3 | root2 | 1 ETH | 0
```

계정 삭제

예시: SELFDESTRUCT

```
address | storage_key | proof_type | new_root | old_root | new_value | old_value
---------|-------------|--------------------|----------|----------|-----------|-
0x9abc.. | 0 | AccountDestructed | root2 | root1 | 0 | balance
```

State Circuit과의 연동

State Circuit은 MPT Table을 조회하여 상태 변경을 검증합니다.

Lookup 패턴

```
// State Circuit에서 MPT Table lookup
meta.lookup_any("mpt proof", |meta| {
vec![
1.expr(), // q_enable
address.expr(), // address
storage_key_rlc.expr(), // storage_key
proof_type.expr(), // proof_type
new_root.expr(), // new_root
old_root.expr(), // old_root
new_value.expr(), // new_value
old_value.expr(), // old_value
]
```

```
});
```

검증 내용

1. **루트 연속성**: 이전 업데이트의 `new_root` 가 다음 업데이트의 `old_root` 와 일치
2. **Merkle 증명**: 각 업데이트에 대한 유효한 Merkle 증명 존재
3. **값 일관성**: RW Table의 값과 MPT Table의 값이 일치

MPT Circuit과의 연동

MPT Circuit은 MPT Table에 데이터를 할당하고 Merkle 증명을 생성합니다.

데이터 할당

파일: `zkevm-circuits/src/table.rs` (line 820)

```
impl MptTable {
pub fn load<F: Field>(
&self,
layouter: &mut impl Layouter<F>,
updates: &MptUpdates,
max_mpt_rows: usize,
randomness: Value<F>,
) -> Result<(), Error> {
layouter.assign_region(
|| "mpt table zkevm",
```

```
|mut region| {
let mpt_update_rows = updates.table_assignments(randomness);
for (offset, row) in mpt_update_rows
.into_iter()
.chain(repeat(MptUpdateRow::padding()))
.take(max_mpt_rows)
.enumerate()
{
self.assign(&mut region, offset, &row)?;
}
Ok(())
},
)
}
}
```

MptUpdates 구조체

파일: `zkevm-circuits/src/witness/mpt.rs`

```
pub struct MptUpdates {
pub updates: Vec<MptUpdate>,
}
```

```
pub struct MptUpdate {
pub address: Address,
pub storage_key: Option<Word>,
pub proof_type: MPTProofType,
pub new_root: Word,
pub old_root: Word,
pub new_value: Word,
pub old_value: Word,
}
```

증명 생성 과정

1. 상태 변경 수집

Bus-Mapping 단계에서 모든 상태 변경을 수집합니다:

```
// 계정 잔액 변경
RwOp::Account {
address: 0x1234,
field_tag: AccountFieldTag::Balance,
value: 100,
value_prev: 50,
}
```

2. MPT 업데이트 생성

각 상태 변경에 대해 MPT 업데이트를 생성합니다:

```
MptUpdate {
address: 0x1234,
storage_key: None,
proof_type: MPTProofType::BalanceChanged,
new_root: compute_new_root(old_root, address, new_value),
old_root: old_root,
new_value: 100,
old_value: 50,
}
```

3. Merkle 증명 생성

MPT Circuit은 각 업데이트에 대한 Merkle 증명을 생성합니다:

```
// zkTrie에서 증명 생성
let  proof = zktrie.prove(address, field_tag);


// 증명 검증
assert!(verify_proof(old_root, address, old_value, proof));
assert!(verify_proof(new_root, address, new_value, proof));
```

4. 테이블 할당

생성된 증명을 MPT Table에 할당합니다.

성능 최적화

1. Poseidon 해시

- SNARK-friendly 해시 함수
- 회로 크기 대폭 감소
- Keccak256 대비 10배 이상 효율적

2. Sparse Binary Tree

- 대부분의 노드가 비어있음
- 메모리 효율적
- 증명 크기 감소

3. SecondPhase 컬럼

RLC를 사용하는 컬럼들은 SecondPhase에 할당:

- `storage_key`
- `new_root`, `old_root`
- `new_value`, `old_value`

4. 병렬 증명 생성

여러 MPT 업데이트를 병렬로 처리하여 성능 향상

제약 조건

1. 루트 연속성

```
// 연속된 업데이트의 루트가 연결되어야 함
update[i].new_root == update[i+1].old_root
```

2. Merkle 증명 유효성

```
// 각 업데이트에 대한 유효한 Merkle 증명
verify_proof(old_root, address, old_value, proof) == true
verify_proof(new_root, address, new_value, proof) == true
```

3. 값 일관성

```
// RW Table의 값과 MPT Table의 값이 일치
rw_table.value == mpt_table.new_value
rw_table.value_prev == mpt_table.old_value
```

예시: 전체 상태 업데이트

Solidity 코드

```
contract Example {
uint256 public value;
function setValue(uint256 newValue) public {
```

```
value = newValue; // 스토리지 업데이트

}

}
```

상태 변경

1. **Nonce 증가** (트랜잭션 발신자)
2. **Balance 감소** (가스 지불)
3. **Storage 변경** (value 업데이트)

MPT Table

```
address | storage_key | proof_type | new_root | old_root | new_value | old_value

---------|-------------|-----------------|----------|----------|-----------|----

sender | 0 | NonceChanged | root2 | root1 | 6 | 5

sender | 0 | BalanceChanged | root3 | root2 | 99 ETH | 100 ETH

contract | key_rlc | StorageChanged | root4 | root3 | 42 | 0
```

루트 연속성:

- root1 → root2 (Nonce 변경)
- root2 → root3 (Balance 변경)
- root3 → root4 (Storage 변경)

디버깅 팁

MPT 업데이트 덤프

```
// MPT 업데이트 출력
for update in mpt_updates.updates.iter() {
println!("MPT Update: address={:?}, proof_type={:?}, old_value={}, new_value={}"
update.address, update.proof_type, update.old_value, update.new_value);
}
```

루트 연속성 검증

```
// 루트 연속성 확인
for i in 0..updates.len() - 1 {
assert_eq!(updates[i].new_root, updates[i+1].old_root,
"Root mismatch at index {}", i);
}
```

일반적인 오류

1. **루트 불일치**: 연속된 업데이트의 루트가 연결되지 않음
2. **증명 실패**: Merkle 증명이 유효하지 않음
3. **값 불일치**: RW Table과 MPT Table의 값이 다름

zkTrie vs 이더리움 MPT

이더리움 MPT

- **해시**: Keccak256
- **구조**: Hexary tree (16개 자식)
- **용도**: 이더리움 메인넷

zkTrie (Scroll)

- **해시**: Poseidon
- **구조**: Binary tree (2개 자식)
- **용도**: ZKEVM 증명
- **장점**: SNARK-friendly, 회로 크기 감소

관련 문서

- State Circuit - MPT Table을 조회하는 회로
- RW Table - 상태 변경 추적
- Poseidon Circuit - Poseidon 해시 회로
- RLC 기법 - RLC 인코딩 상세

참고 자료

- **코드**: `zkevm-circuits/src/table.rs` (line 788)
- **MPT Circuit**: `mpt-circuits/`
- **zkTrie**: `zktrie/`
- **Poseidon**: https://www.poseidon-hash.info/
- **이더리움 Yellow Paper**: https://ethereum.github.io/yellowpaper/paper.pdf (Appendix D)